

✓ Exercises XP: RAG with LangChain (Student)

✓ 0) Setup

```
!pip -q install -U datasets transformers sentence-transformers faiss-
```

```
████████████████████████████████████████████████████████████████████████████████ 529.0/529.0 kB 16.5 MB/s eta 0s
████████████████████████████████████████████████████████████████████████████████ 10.6/10.6 MB 59.8 MB/s eta 0s
████████████████████████████████████████████████████████████████████████████████ 23.8/23.8 MB 33.2 MB/s eta 0s
████████████████████████████████████████████████████████████████████████████████ 113.6/113.6 kB 5.0 MB/s eta 0s
████████████████████████████████████████████████████████████████████████████████ 543.9/543.9 kB 16.6 MB/s eta 0s
████████████████████████████████████████████████████████████████████████████████ 2.5/2.5 MB 29.5 MB/s eta 0s
████████████████████████████████████████████████████████████████████████████████ 1.0/1.0 MB 19.9 MB/s eta 0s
████████████████████████████████████████████████████████████████████████████████ 173.8/173.8 kB 5.0 MB/s eta 0s
████████████████████████████████████████████████████████████████████████████████ 48.9/48.9 MB 14.3 MB/s eta 0s
████████████████████████████████████████████████████████████████████████████████ 64.9/64.9 kB 3.5 MB/s eta 0s
████████████████████████████████████████████████████████████████████████████████ 51.0/51.0 kB 2.5 MB/s eta 0s
```

```
ERROR: pip's dependency resolver does not currently take into account
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.31.0
```

```
from typing import List
```

```
from datasets import load_dataset
from transformers import pipeline
```

```
from langchain_core.documents import Document
from langchain_core.prompts import PromptTemplate
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_community.vectorstores.utils import DistanceStrategy
```

```
from langchain_huggingface import HuggingFacePipeline
from langchain_classic.chains import RetrievalQA
```

✓ 1) Load dataset and convert to Documents

```
dataset_name = "m-ric/huggingface_doc"
split = "train[:200]"
text_column = "text"
source_column = "source"
```

```
ds = load_dataset(dataset_name, split=split)
```

```
documents: List[Document] = []
```

```

for i, row in enumerate(ds):
    documents.append(
        Document(
            page_content=row[text_column],
            metadata={"source": row[source_column]}
        )
    )

print("Documents:", len(documents))
print("Example:", documents[0].metadata)
print(documents[0].page_content[:350])

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your sett
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to a
  warnings.warn(

README.md: 100%                               21.0/21.0 [00:00<00:00, 279B/s]
Warning: You are sending unauthenticated requests to the HF Hub. Pleas
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthent
huggingface_doc.csv: 100%                      22.0M/22.0M [00:01<00:00, 110MB/s]

Generating train split: 100%                   2647/2647 [00:00<00:00, 2834.35 examples/s]
Documents: 200
Example: {'source': 'huggingface/hf-endpoints-documentation/blob/main/
  Create an Endpoint

After your first login, you will be directed to the [Endpoint creation

```

✓ 2) Split into chunks

```

chunk_size = 512
chunk_overlap = 50

splitter = RecursiveCharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap,
    add_start_index=True,
    strip_whitespace=True
)

splits = splitter.split_documents(documents)

print("Chunks:", len(splits))
print("First chunk metadata:", splits[0].metadata)
print("-" * 30)
print("First chunk content:\n", splits[0].page_content[:350])

```

```

Chunks: 5789
First chunk metadata: {'source': 'huggingface/hf-endpoints-documentati
-----
First chunk content:
  Create an Endpoint

After your first login, you will be directed to the [Endpoint creation

```

✓ 3) Vector store + retriever (FAISS)

```

from langchain_community.vectorstores import FAISS
from langchain_community.vectorstores.utils import DistanceStrategy

embedding_model = "sentence-transformers/all-MiniLM-L6-v2"
embeddings = HuggingFaceEmbeddings(model_name=embedding_model)

vectorstore = FAISS.from_documents(
    documents=splits,
    embedding=embeddings,
    distance_strategy=DistanceStrategy.COSINE
)

retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
print("Retriever ready")

```

```

/tmp/ipykernel_18808/912447508.py:5: LangChainDeprecationWarning: The
  embeddings = HuggingFaceEmbeddings(model_name=embedding_model)

modules.json: 100% 349/349 [00:00<00:00, 24.2kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 9.21kB/s]

README.md: 10.5k/? [00:00<00:00, 687kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 5.29kB/s]

config.json: 100% 612/612 [00:00<00:00, 29.1kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 453MB/s]

Loading weights: 100% 103/103 [00:00<00:00, 2017.70it/s]

tokenizer_config.json: 100% 350/350 [00:00<00:00, 24.6kB/s]

vocab.txt: 232k/? [00:00<00:00, 6.94MB/s]

tokenizer.json: 466k/? [00:00<00:00, 19.6MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 8.68kB/s]

config.json: 100% 190/190 [00:00<00:00, 13.3kB/s]

Retriever ready

```

✓ 4) Build the RAG chain

```
!pip install langchain
```

```

Requirement already satisfied: langchain in /usr/local/lib/python3.12/
Requirement already satisfied: langchain-core<2.0.0,>=1.3.3 in /usr/lo
Requirement already satisfied: langgraph<1.2.0,>=1.1.10 in /usr/local/
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/li
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/
Requirement already satisfied: langchain-protocol>=0.0.10 in /usr/loca
Requirement already satisfied: langsmith<1.0.0,>=0.3.45 in /usr/local/
Requirement already satisfied: packaging>=23.2.0 in /usr/local/lib/pyt
Requirement already satisfied: pyyaml<7.0.0,>=5.3.0 in /usr/local/lib/
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in /usr
Requirement already satisfied: uuid-utils<1.0,>=0.12.0 in /usr/local/l
Requirement already satisfied: langgraph-checkpoint<5.0.0,>=2.1.0 in /
Requirement already satisfied: langgraph-prebuilt<1.1.0,>=1.0.12 in /u
Requirement already satisfied: langgraph-sdk<0.4.0,>=0.3.0 in /usr/loc
Requirement already satisfied: xxhash>=3.5.0 in /usr/local/lib/python3
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/li
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/pyth
Requirement already satisfied: ormsgpack>=1.12.0 in /usr/local/lib/pyt
Requirement already satisfied: httpx>=0.25.2 in /usr/local/lib/python3
Requirement already satisfied: orjson>=3.11.5 in /usr/local/lib/python
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/
Requirement already satisfied: requests>=2.0.0 in /usr/local/lib/pytho

```

```
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/pyt
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist
Requirement already satisfied: certifi in /usr/local/lib/python3.12/di
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/
Requirement already satisfied: urllib3<3,>=1.26 in /usr/local/lib/pyth
```

```
!pip install -U langchain-classic
```

```
Requirement already satisfied: langchain-classic in /usr/local/lib/pyt
Requirement already satisfied: langchain-core<2.0.0,>=1.3.3 in /usr/lo
Requirement already satisfied: langchain-text-splitters<2.0.0,>=1.1.2
Requirement already satisfied: langsmith<1.0.0,>=0.1.17 in /usr/local/
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/li
Requirement already satisfied: pyyaml<7.0.0,>=5.3.0 in /usr/local/lib/
Requirement already satisfied: requests<3.0.0,>=2.0.0 in /usr/local/li
Requirement already satisfied: sqlalchemy<3.0.0,>=1.4.0 in /usr/local/
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/
Requirement already satisfied: langchain-protocol>=0.0.10 in /usr/loca
Requirement already satisfied: packaging>=23.2.0 in /usr/local/lib/pyt
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in /usr
Requirement already satisfied: uuid-utils<1.0,>=0.12.0 in /usr/local/l
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/pyth
Requirement already satisfied: orjson>=3.9.14 in /usr/local/lib/python
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/
Requirement already satisfied: xxhash>=3.0.0 in /usr/local/lib/python3
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/pyt
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/li
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.
Requirement already satisfied: urllib3<3,>=1.26 in /usr/local/lib/pyth
Requirement already satisfied: certifi>=2023.5.7 in /usr/local/lib/pyt
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.1
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/pyth
```

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, pipeli
from langchain_huggingface import HuggingFacePipeline
from langchain_classic.chains import RetrievalQA
```

```
llm_id = "google/flan-t5-small"
```

```
tokenizer = AutoTokenizer.from_pretrained(llm_id)
model = AutoModelForSeq2SeqLM.from_pretrained(llm_id)
```

```
hf = pipeline(
    task="text-generation",
    model=model,
    tokenizer=tokenizer,
```

```

        device_map="auto",
        max_new_tokens=256
    )

    llm = HuggingFacePipeline(pipeline=hf)

    qa = RetrievalQA.from_chain_type(
        llm=llm,
        chain_type="stuff",
        retriever=retriever
    )

    print("RAG chain ready")

```

```

Loading weights: 100% 190/190 [00:00<00:00, 2339.79it/s]
[transformers] The tied weights mapping and config for this model spec
[transformers] Passing `generation_config` together with generation-re
[transformers] The model 'T5ForConditionalGeneration' is not supported
RAG chain ready

```

```

question = "How can I use the Hugging Face hub?"
response = qa.invoke(question)
print(response["result"])

```

```

[transformers] Both `max_new_tokens` (=256) and `max_length` (=20) seem
Use the following pieces of context to answer the question at the end.

```

Joining Hugging Face and installation

To share models in the Hub, you will need to have a user. Create it on
 This integration is made possible by the [`huggingface_hub`](https://github.com/huggingface/huggingface_hub)
 A shout out to the Hugging Face team for all the work on this integration
 Thank you fastlearners and hugging learners 🥰.

First, log in to the Hugging Face Hub. You will need to create a `writer` user.
 1. Type `huggingface-cli login` in your terminal and enter your token.
 2. If in a python notebook, you can use `notebook_login`.

```

```py
from huggingface_hub import notebook_login

notebook_login()
```

```

3. Use the `token` argument of the `push_to_hub_fastai` function.

Setup

Before sharing a model to the Hub, you will need your Hugging Face credentials.

```
```bash
huggingface-cli login
```
```

Question: How can I use the Hugging Face hub?
Helpful Answer:

✓ 5) Demo: RAG vs no-RAG

```
q = "How can I retrieve a model from the Hugging Face Hub?"

no_rag_prompt = (
    "Answer the question. If you are not sure, say you are not s"
    f"Question: {q}\n"
    "Answer:"
)
no_rag_answer = hf(no_rag_prompt)[0]["generated_text"]

rag_result = qa.invoke({"query": q})

print("Q:", q)
print("\n--- No-RAG answer ---\n", no_rag_answer)
print("\n--- RAG answer ---\n", rag_result["result"])
print("\nSources:")

if "source_documents" in rag_result:
    for d in rag_result["source_documents"]:
        print("-", d.metadata.get("source"))
else:
    print("No sources returned.")
```

```
[transformers] Both `max_new_tokens` (=256) and `max_length` (=20) seem
[transformers] Both `max_new_tokens` (=256) and `max_length` (=20) seem
Q: How can I retrieve a model from the Hugging Face Hub?
```

```
--- No-RAG answer ---
Answer the question. If you are not sure, say you are not sure.
```

```
Question: How can I retrieve a model from the Hugging Face Hub?
Answer:
```

```
--- RAG answer ---
Use the following pieces of context to answer the question at the end

## Joining Hugging Face and installation

To share models in the Hub, you will need to have a user. Create it on
Now when you navigate to your Hugging Face profile, you should see you
For more details on how to create and upload files to a repository, re
## Upload with the web interface
```

This integration is made possible by the [`huggingface_hub`](https://github.com/huggingface/huggingface_hub).

A shout out to the Hugging Face team for all the work on this integration.

Thank you fastlearners and hugging learners 😊.

Setup

Before sharing a model to the Hub, you will need your Hugging Face credentials.

```
```bash
huggingface-cli login
```
```

Question: How can I retrieve a model from the Hugging Face Hub?
Helpful Answer:

Sources:
No sources returned.

6. Retrieval sanity check (required)

```
test_query = "What is the command to log into the Hugging Face H

retrieved_chunks = retriever.invoke(test_query)

print(f"--- Sanity Check for Query: '{test_query}' ---\n")
print(f"Number of chunks retrieved: {len(retrieved_chunks)}")

for i, doc in enumerate(retrieved_chunks):
    print(f"\n[Chunk {i+1}]")
    print(f"Source: {doc.metadata.get('source')}")
    print(f"Content: {doc.page_content[:300]}...")
    print("-" * 20)
```

```
--- Sanity Check for Query: 'What is the command to log into the Huggi

Number of chunks retrieved: 4
```

```
[Chunk 1]
Source: huggingface/blog/blob/main/fastai.md
Content: First, log in to the Hugging Face Hub. You will need to creat

1. Type `huggingface-cli login` in your terminal and enter your token.

2. If in a python notebook, you can use `notebook_...
-----
```

```
[Chunk 2]
Source: huggingface/huggingface_hub/blob/main/docs/source/en/guides/cl
Content: # Command Line Interface (CLI)
```

```
The `huggingface_hub` Python package comes with a built-in CLI called
-----
```

```
[Chunk 3]
```

Source: huggingface/huggingface_hub/blob/main/docs/source/en/guides/cli
Content: </Tip>

```
## huggingface-cli login
```

In many cases, you must be logged in to a Hugging Face account to inte

[Chunk 4]

Source: huggingface/huggingface_hub/blob/main/docs/source/en/guides/cli
Content: ## huggingface-cli whoami

If you want to know if you are logged in, you can use `huggingface-cli

```
```bash
huggingface-cli whoami

```

## 7. RAG question answering

```
test_questions = [
 "How do I log into the Hugging Face Hub in a notebook?",
 "What library makes the Hugging Face integration possible?",
 "How can I share models in the Hub?"
]

print("--- Starting RAG Evaluation ---\n")

for i, question in enumerate(test_questions):
 print(f"Test {i+1}: {question}")
 result = qa.invoke({"query": question})

 print(f"Response: {result['result']}")

 print("Sources detected:")
 sources = set([doc.metadata.get("source") for doc in result])
 for s in sources:
 print(f" - {s}")

 print("-" * 30 + "\n")

print("Exercise XP Complete!")
```

and [ Accelerate](<https://github.com/huggingface/accelerate>) – as v

and for JavaScript, the [HuggingFace.js library](<https://huggingface.js>

For more information on the Hugging Face paths and how they are imple

Question: What library makes the Hugging Face integration possible?

Helpful Answer:

Sources detected:

-----

Test 3: How can I share models in the Hub?

[transformers] Both `max\_new\_tokens` (=256) and `max\_length` (=20) see

Response: Use the following pieces of context to answer the question

## Why share to the Hub?

The Hub is a central platform where anyone can share and explore mode

Sharing on the Hub amplifies the impact of your fastai models by maki

Sharing a model to the Hub is as simple as adding an extra parameter

```
```py
>>> training_args = TrainingArguments(output_dir="my-awesome-model",
```
```

## Joining Hugging Face and installation

To share models in the Hub, you will need to have a user. Create it o

## Setup

Before sharing a model to the Hub, you will need your Hugging Face cr

```
```bash
huggingface-cli login
```
```